TP AVANCÉ

Pipeline CI/CD intelligent & analyse de qualité / risque applicatif

PayFlex API — FastAPI, Docker, PyTest, Newman, k6, SonarQube et Gitleaks

OBJECTIF	GARANTIE PRINCIPALE
Automatiser la validation et le déploiement d'une API financière.	Aucune erreur critique, baisse de performance ou fuite ne peut atteindre la production.

Rapport de synthèse technique

Version préparée le 11 novembre 2026. Le document décrit le pipeline implémenté et la procédure de preuve des trois scénarios demandés.

1. Architecture générale

Le workflow est une chaîne séquentielle. Chaque job dépend du précédent avec **needs**. Un échec critique arrête donc la progression avant le déploiement.

Toute erreur critique produit un rapport d'incident et empêche les étapes suivantes.

Principe de l'artefact unique

L'image Docker est construite une seule fois dans le premier job, puis exportée comme artefact. Newman, k6 et le déploiement chargent exactement cette image identifiée par le SHA du commit. Le code testé est ainsi identique au code publié.

Déclencheurs

• **push** : exécute toute la CI ; un push sur main peut publier l'image.

• **pull_request** : valide le changement sans déployer.

• **workflow_dispatch** : permet d'exécuter les trois scénarios de démonstration.

Fail-fast

Un code de sortie non nul marque le job en échec. GitHub Actions ignore alors tous ses descendants. Les étapes marquées **if: always()** conservent néanmoins les logs, métriques et rapports d'incident.

2. Confidentialité, tests et qualité

Étape 1 — Gitleaks et construction

Gitleaks analyse l'historique Git, les sources/configurations suivies et le contenu applicatif de l'image Docker. Les règles standards sont complétées par des expressions de détection pour les cartes bancaires et les IBAN.

Le seuil maximal toléré étant zéro, la formule fournie dans l'énoncé est remplacée par une convention binaire : **0 fuite = 100 %** ; **au moins 1 fuite = 0 %**. Un rapport absent ou illisible vaut également 0 %, afin d'éviter qu'une panne du scanner soit interprétée comme un succès.

Étape 2 — PyTest et couverture

Les tests unitaires couvrent le health check, la création, la validation métier, les champs obligatoires et la consultation d'un paiement. PyTest produit coverage.xml, junit.xml et pytest.log. Le Quality Gate classe ensuite la couverture selon le tableau des seuils.

Étape 3 — SonarQube Community

Une instance éphémère est démarrée dans le runner. Son Quality Gate bloque une couverture inférieure à 70 %, tout bug ou toute vulnérabilité. Les code smells et la zone de couverture 70–80 % restent non critiques et génèrent un warning.

Indicateur	Conforme	Warning	Critique
Couverture	>= 80 %	70 à < 80 %	< 70 %
Latence p95	<= 200 ms	> 200 à <= 500 ms	> 500 ms
Taux d'erreur	< 1 %	>= 1 à <= 2 %	> 2 %
Bugs / vulnérabilités	0	Code smells	> 0
Confidentialité	100 %	—	< 100 %

Règle de décision

VERT : continuer. ORANGE : continuer avec alerte. ROUGE : rapport d'incident, code de sortie 1 et aucun déploiement.

3. Intégration, performance et déploiement

Étape 4 — Docker + Newman

Le job charge l'image validée et démarre un conteneur éphémère. Un health check confirme la disponibilité de l'API, puis Newman vérifie les contrats HTTP et le flux création/lecture. Les cas d'erreur 400 et 404 sont aussi contrôlés. Un crash ou une assertion Newman échouée est critique.

Étape 5 — k6

k6 applique 20 utilisateurs virtuels pendant 30 secondes. Il mesure le p95, les erreurs HTTP et les checks. Le taux d'erreur effectif est la valeur la plus défavorable entre erreurs HTTP et checks en échec. Les seuils critiques sont présents dans k6 et réévalués par quality_gate.py pour produire une classification claire.

Étape 6 — publication contrôlée

Le job de déploiement ne s'exécute que sur un push réussi vers main. Il charge l'image déjà testée, la marque avec le SHA du commit puis la publie dans GitHub Container Registry. Une branche, une Pull Request ou un lancement manuel ne peut pas publier.

Analyse des logs

Après PyTest, Newman et k6, les sorties et les logs applicatifs sont scannés par Gitleaks. Une fuite dans un log est soumise au même seuil de confidentialité de 100 % que le code et l'artefact.

Rapports produits

Job	Rapports principaux
Confidentialité	gitleaks-*.json, quality-report-secrets.json
Unitaires	coverage.xml, junit.xml, pytest.log
SonarQube	sonar-measures.json, quality-report-sonar.json
Intégration	newman-report.json, logs applicatifs
Performance	k6-summary.json, k6.log, logs applicatifs
Erreur critique	incident-*.md + résumé GitHub Actions

4. Classification automatique des erreurs

Le script quality_gate.py lit les rapports techniques, normalise les métriques et produit un rapport JSON contenant stage, status, metrics, warnings et critical_errors.

Erreur non critique

• Couverture comprise entre 70 % et 80 %.

• Latence p95 comprise entre 200 ms et 500 ms.

• Taux d'erreur compris entre 1 % et 2 %.

• Code smells sans bug ni vulnérabilité.

Le pipeline continue, mais une annotation warning et un résumé sont générés.

Erreur critique

• Test unitaire, Newman ou démarrage de l'API en échec.

• Couverture inférieure à 70 %, bug ou vulnérabilité SonarQube.

• p95 supérieur à 500 ms ou taux d'erreur supérieur à 2 %.

• Fuite dans les sources, l'historique, l'artefact ou les logs.

• Rapport obligatoire absent ou invalide.

Le script écrit incident-*.md, publie des erreurs GitHub et termine avec le code 1. La dépendance needs bloque alors immédiatement la suite.

Scénarios de preuve intégrés

Entrée	Effet	Résultat attendu
success	Aucune injection	Pipeline vert
performance_failure	Latence de test à 600 ms	Échec au job k6
secret_leak	PAN de test créé à l'exécution	Échec au premier job

Les données de démonstration sont synthétiques et ne contiennent aucun secret réel.

5. Captures d'écran des exécutions

Les emplacements ci-dessous doivent recevoir les captures réelles après publication du workflow corrigé. Dans GitHub Actions, lancer success, performance_failure puis secret_leak depuis Run workflow.

Capture 1 — Pipeline réussi

Tous les jobs de validation verts ; déploiement ignoré lors du lancement manuel.

Insérer ici la capture GitHub Actions après l'exécution du workflow corrigé

Capture 2 — Échec de performance

Job performance rouge ; p95 > 500 ms ; job deploy absent ou skipped.

Insérer ici la capture GitHub Actions après l'exécution du workflow corrigé

Capture 3 — Fuite confidentielle

Job confidentialité rouge ; taux 0 % ; jobs suivants skipped.

Insérer ici la capture GitHub Actions après l'exécution du workflow corrigé

Note : les captures ne peuvent être produites qu'après un push et une exécution distante. Elles ne sont pas simulées dans ce rapport afin de préserver la valeur probante du livrable.

6. Vérification du livrable

Exigence	Implémentation	État
Tests unitaires + couverture	PyTest, coverage.xml, seuils 70/80	OK
Tests d'intégration	Docker éphémère + Newman	OK
Tests de performance	k6, p95 et taux d'erreur	OK
Qualité statique	SonarQube Community + Quality Gate	OK
Confidentialité	Gitleaks sources, artefact, logs, PII	OK
Fail-fast	needs + codes de sortie critiques	OK
Rapports d'incident	JSON, Markdown, artefacts always()	OK
Déploiement conditionnel	Image validée vers GHCR sur main	OK
Captures GitHub Actions	Trois exécutions workflow_dispatch	À insérer

Conclusion

Le pipeline PayFlex couvre la chaîne demandée de bout en bout : construction immuable, tests multi-niveaux, analyse qualité, confidentialité à tolérance zéro, classification automatique et publication uniquement après validation complète. Les avertissements restent visibles sans interrompre le flux ; toute anomalie critique interdit le déploiement.